cadence

Cadence Driver Porting Guide

Product Version 1.4

September 2017

© 1996-2017 Cadence Design Systems, Inc. All rights reserved.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 800.862.4522. All other trademarks are the property of their respective holders.

Restricted Permission: This document is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this document, or any portion of it, may result in civil and criminal penalties. Except as specified in this permission statement, this document may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This document contains the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only in accordance with, a written agreement between Cadence and its customer.

Unless otherwise agreed to by Cadence in writing, this statement grants Cadence customers permission to print one (1) hard copy of this document subject to the following conditions:

1. This document may not be modified in any way.

2. Any authorized copy of this document or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement.

3. The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration.

Disclaimer: INFORMATION IN THIS DOCUMENT IS SUBJECT TO CHANGE WITHOUT NOTICE AND DOES NOT REPRESENT A COMMITMENT ON THE PART OF CADENCE. EXCEPT AS MAY BE EXPLICITLY SET FORTH IN A WRITTEN AGREEMENT BETWEEN CADENCE AND ITS CUSTOMER, CADENCE DOES NOT MAKE, AND EXPRESSLY DISCLAIMS, ANY REPRESENTATIONS OR WARRANTIES AS TO THE COMPLETENESS, ACCURACY OR USEFULNESS OF THE INFORMATION CONTAINED IN THIS DOCUMENT. CADENCE DOES NOT WARRANT THAT USE OF SUCH INFORMATION WILL NOT INFRINGE ANY THIRD PARTY RIGHTS, AND CADENCE DISCLAIMS ALL IMPLIED WARRANTIES, INCLUDING MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. CADENCE DOES NOT ASSUME ANY LIABILITY FOR DAMAGES OR COSTS OF ANY KIND THAT MAY RESULT FROM USE OF SUCH INFORMATION. CADENCE CUSTOMER HAS COMPLETE CONTROL AND FINAL DECISION-MAKING AUTHORITY OVER ALL ASPECTS OF THE DEVELOPMENT, MANUFACTURE, SALE AND USE OF CUSTOMER'S PRODUCT, INCLUDING, BUT NOT LIMITED TO, ALL DECISIONS WITH REGARD TO DESIGN, PRODUCTION, TESTING, ASSEMBLY, QUALIFICATION, CERTIFICATION, INTEGRATION OF CADENCE PRODUCTS, INSTRUCTIONS FOR USE, LABELING AND DISTRIBUTION, AND CADENCE EXPRESSLY DISAVOWS ANY RESPONSIBILITY WITH REGARD TO ANY SUCH DECISIONS REGARDING CUSTOMER'S PRODUCT.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227- 14 and DFAR252.227-7013 et seq. or its successor.

Cadence Driver Porting Guide

Cadence Design Systems

Table of Contents

1. Acronyms 1
2. Overview 2
    2.1. Document Purpose 2
    2.2. CPS Overview 2
    2.3. Common Driver Interface Overview 3
    2.4. Use Model 3
        2.4.1. Bare-Metal Architecture 3
        2.4.2. Microkernel Architecture 4
        2.4.3. Full OS Architecture 5
    2.5. Concurrency Models 6
3. Common Driver Interface 8
    3.1. Overview 8
    3.2. Driver Initialization & Startup 8
    3.3. Concurrency Model Details 9
4. Cadence Platform Services Reference 12
    4.1. Introduction 12
    4.2. CPS_LockHandle 12
    4.3. CPS_ProbeLocks 12
    4.4. CPS_InitLock 12
    4.5. CPS_FreeLock 13
    4.6. CPS_Lock 13
    4.7. CPS_Unlock 13
    4.8. CPS_UncachedRead8 14
    4.9. CPS_UncachedRead16 14
    4.10. CPS_UncachedRead32 14
    4.11. CPS_UncachedWrite8 15
    4.12. CPS_UncachedWrite16 15
    4.13. CPS_UncachedWrite32 16
    4.14. CPS_WritePhysAddress32 16
    4.15. CPS_BufferCopy 16
    4.16. CPS_CacheInvalidate 17
    4.17. CPS_CacheFlush 17
5. Sample CPS implementation for Bare Metal 19
    5.1. Overview 19
    5.2. Sample code 19
6. Sample CPS implementation for Linux 21
    6.1. Overview 21
    6.2. Sample code 21

List of Figures

2.1. Pictorial Representation of CPS 2
2.2. Bare-Metal Architecture 4
2.3. Microkernel Architecture 5
2.4. Full OS Architecture 6
3.1. Driver Initialization & Startup Sequence 9
3.2. Pure IRQ Concurrency Model 9
3.3. Single Worker Concurrency Model 10
3.4. Multiple Workers Concurrency Model 11

Chapter 1. Acronyms

API	Application Programming Interface.
Core Driver	Cadence Firmware component that provides IP programming abstraction.
CPS	Cadence Platform Services. A set of basic platform-specific access functions acting as hardware abstraction layer for the core driver to operate.

Chapter 2. Overview

2.1. Document Purpose

The purpose of this document is to outline how to perform the following porting tasks related to Cadence drivers:

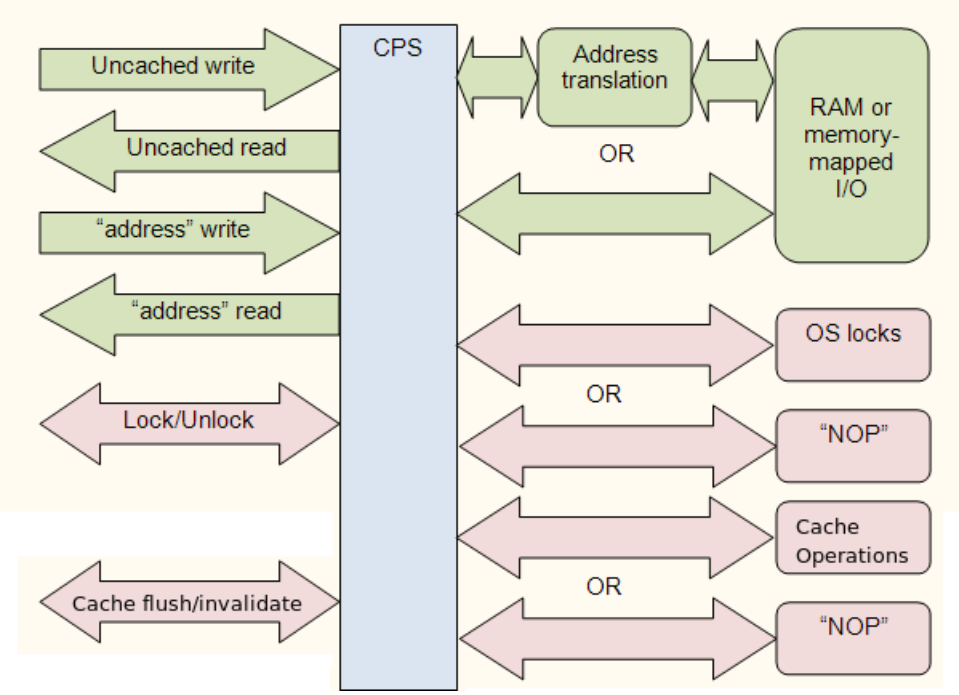
- Implementing CPS for your platform. This need be done only once to enable all Cadence drivers for your platform.
- Implementing the Common Driver Interface for your platform. This is the platform-specific code to connect a given Cadence driver to the rest of your system. Depending on your platform, aspects of this will be specific to the platform or the layer above the driver.

2.2. CPS Overview

Cadence Core Drivers are built on a common hardware abstraction layer, called Cadence Platform Services (CPS). This is where all aspects specific to a CPU are handled. Specifically, the following is handled by CPS:

- Uncached access (i.e. writing through the cache with order preserved) to memory, used for hardware registers and DMA descriptors.
- Translating addresses (if needed) between the CPU's view (when running the driver), and hardware's view. This is used for things such as DMA descriptors, where a buffer address as seen by hardware may be different than that seen by the CPU (e.g. if an MMU is being used).
- Locking access to critical data (if needed), for example if multiple threads or CPUs are being used. Mutex-style locking is assumed.
- Cache flush/invalidate operations (if needed), for example if the hardware needs to access memory directly for DMA operations.

Figure 2.1. Pictorial Representation of CPS



Each platform requires an implementation of CPS. Once CPS is implemented for a platform, all Cadence Core Drivers can use it on that platform. This reduces porting effort in case you acquire further drivers from Cadence.

2.3. Common Driver Interface Overview

Several important aspects of using a driver depend heavily on the platform:

- How the RAM used by the driver must be allocated. On a simple system, it may be sufficient to reserve a block of RAM statically, while on a full memory-protected OS, it may be necessary to use a kernel allocation function.
- How the memory-mapped I/O range for the hardware's registers is obtained. On a simple system, it may be hard-wired, while on other systems it may be necessary to call a mapping function. If the hardware is on a plug-and-play bus such as PCI, the bus enumerator may assign this, requiring a fundamentally different way of connecting the driver.
- How the interrupt number/vector for the hardware is obtained. On a simple system, it may be hard-wired, while on other systems it may be necessary to call a mapping function. If the hardware is on a plug-and-play bus such as PCI, the bus enumerator may assign this, requiring a fundamentally different way of connecting the driver.

Due to the above considerations, for maximum portability:

- Cadence drivers never allocate memory - this must be done by your integration code and passed to the driver.
- Cadence drivers never assume a location for the underlying hardware's registers - this must be determined by your integration code and passed to the driver.
- Cadence drivers never assume a particular interrupt "topology" - your integration code must arrange for the driver's ISR to be called upon receipt of the underlying hardware's interrupt.

This is the Common Driver Interface.

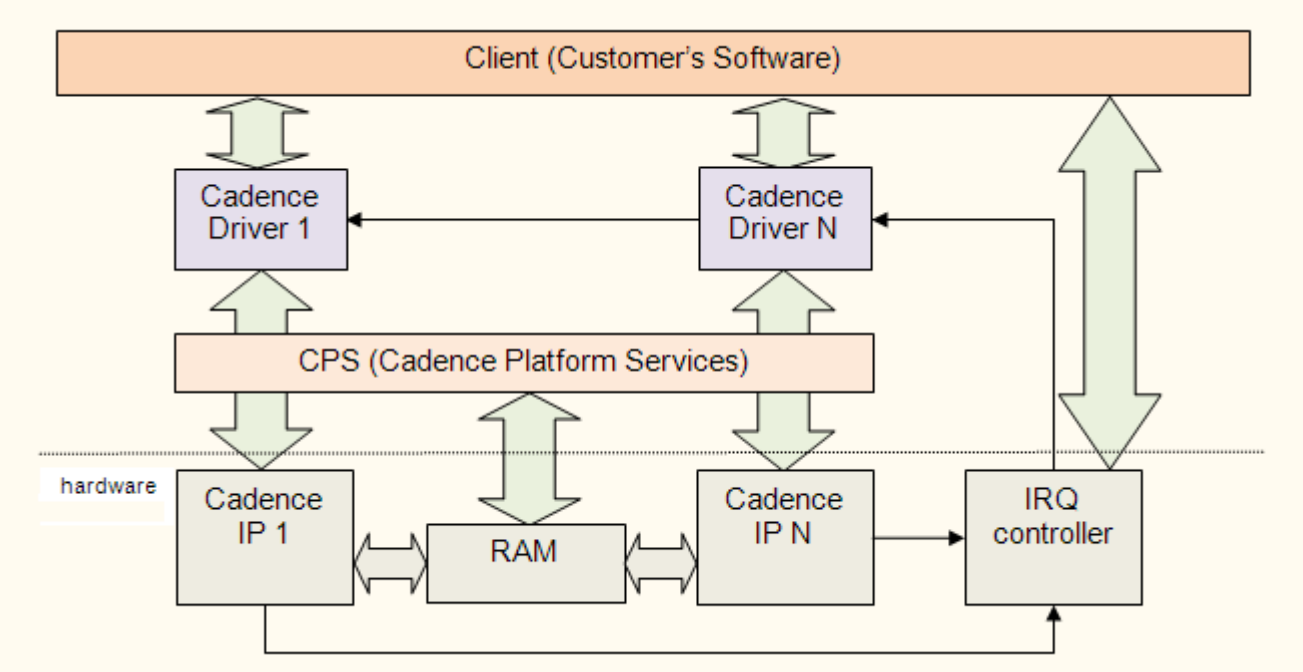
2.4. Use Model

Choosing any aspect of a computing platform, one can find a spectrum of possible implementations. It is simply not possible to list every situation. Instead, the following three classifications of systems are intended to serve as a rough guideline for the tasks required to enable a Cadence driver for your platform.

2.4.1. Bare-Metal Architecture

The following depicts the typical scenario for a simple "bare-metal" platform (a platform not using an MMU, without a formalized porting layer or technology stacks):

Figure 2.2. Bare-Metal Architecture



In such a system, it is likely that your custom software will use the driver API directly, and you may have direct control over the interrupt controller setup. Since there is no MMU it is likely that there will be no data cache available. Implementing CPS on such a platform will typically involve:

- Using simple constructs such as

  ```
  *(volatile uint32_t*)address = value
  ```

  for the memory-mapped I/O functions

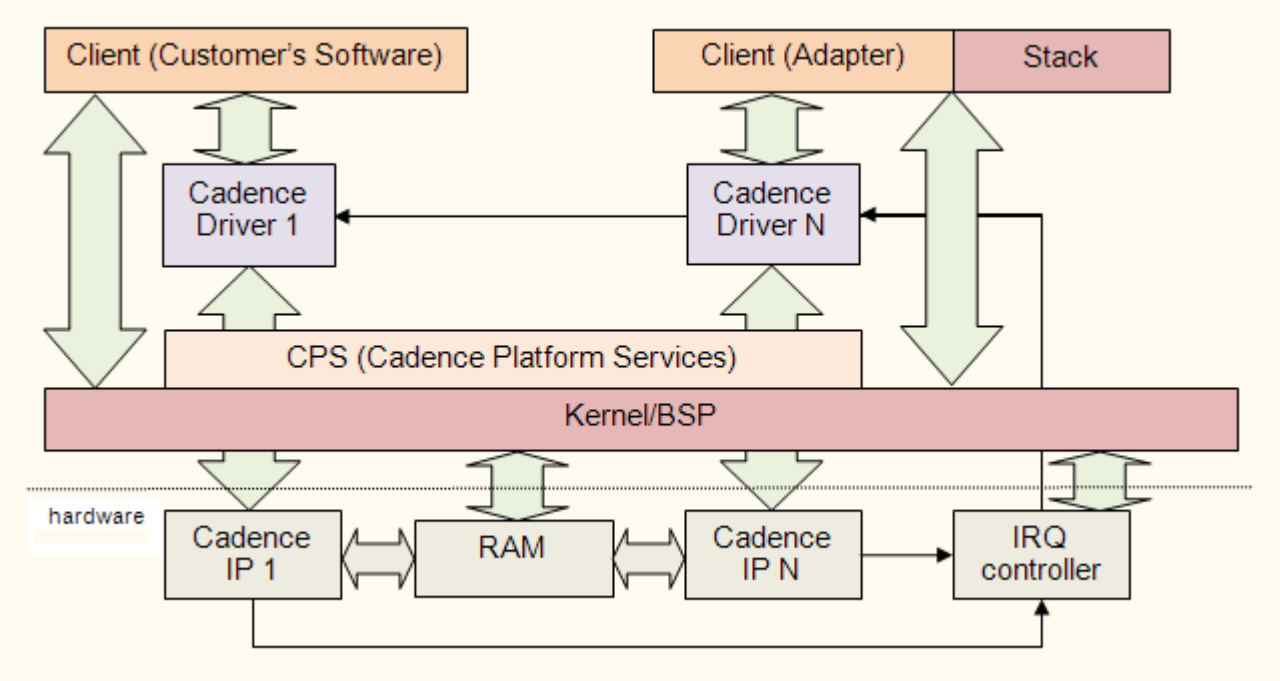
- Using the same for the address-translation functions
- Implementing do-nothing functions for locking
- Implementing do-nothing functions for cache flush and invalidate

Implementing the integration code on such a platform will generally be straightforward unless the hardware is on a plug-and-play bus.

2.4.2. Microkernel Architecture

If you are using an RTOS or microkernel, there may or may not be a standard stack for a given technology. If there is a standard stack, it is likely you would want to create a small adapter layer to convert between the Cadence driver's API and the lowest layer of that stack. We will again assume no MMU is being used on such a platform. Also on such a platform, it is likely that there is some level of support from the kernel or BSP for programming the interrupt controller attaching handlers, and possibly memory-mapped I/O services exist.

Figure 2.3. Microkernel Architecture



Implementing CPS on such a platform will typically involve:

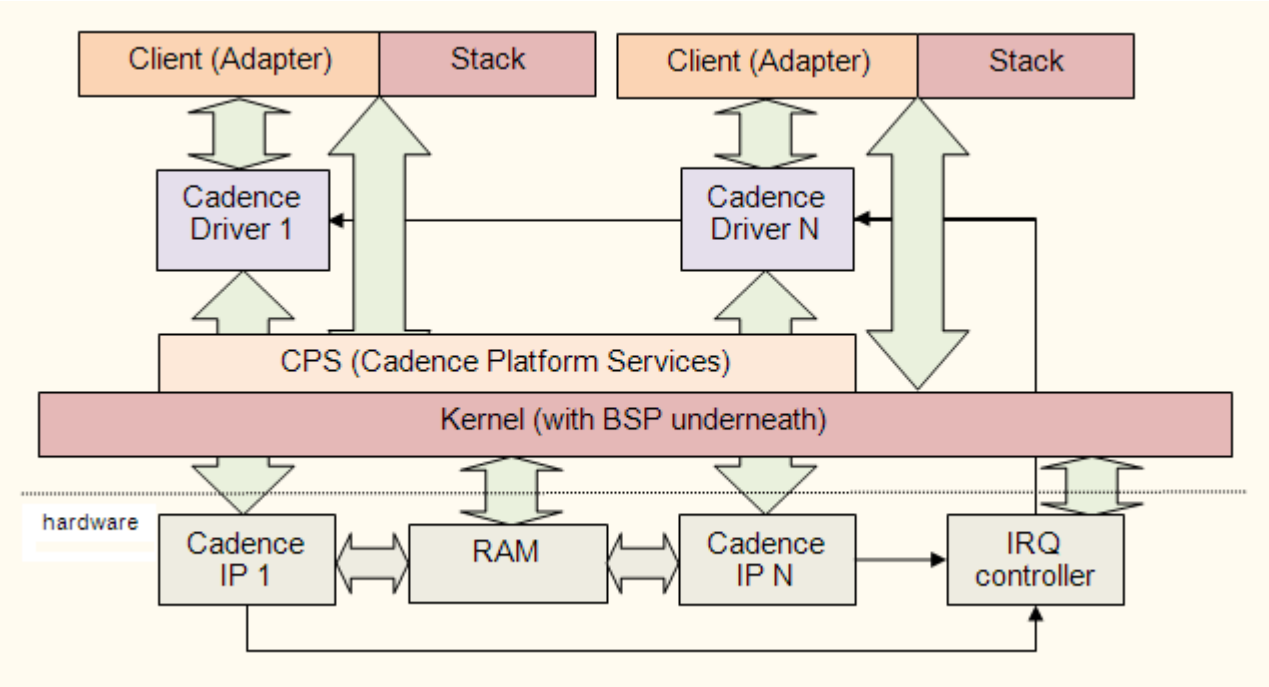
- For memory-mapped I/O, either using simple constructs as in the bare-metal example, or calling service functions in the kernel or BSP
- Using the same for the address-translation functions
- Implementing do-nothing functions for locking, unless a multiple-worker concurrency model is desired, in which case the OS locks should be used
- Implementing do-nothing functions for cache flush and invalidate

Implementing the integration code on such a platform will generally be straightforward unless the hardware is on a plug-and-play bus.

2.4.3. Full OS Architecture

If you are using a full memory-protected OS (definitely an MMU being used), there will typically be a standard stack for a given technology. Therefore, you will likely want to create a small adapter layer to convert between the Cadence driver's API and the lowest layer of that stack. Also on such a platform, it is typical that interrupt attachment and memory-mapped I/O access will be abstracted for you.

Figure 2.4. Full OS Architecture



Implementing CPS on such a platform will typically involve:

- For memory-mapped I/O, calling service functions in the kernel. This usually involves mapping the hardware's addressable region at driver initialization and then using it, either directly or through I/O services, in other driver functions.
- For the address-translation functions, calling translation functions in the kernel
- For locking, this depends on whether a multiple-worker concurrency model is desired (or an equivalent is enforced by the OS)
- Implementing cache flush/invalidate functions using the OS if the hardware accesses memory directly, e.g. for DMA.

On this type of platform, it will be necessary to wrap the Cadence driver to create a driver package which follows the rules of the OS driver framework. This usually means:

- Creating an initialization function conforming to the framework's rules, in which you acquire resources and call the Cadence driver's init function. All Cadence drivers provide probe and init functions to aid in resource acquisition.
- Creating other functions to start, stop and destroy the driver, again calling the corresponding functions in the Cadence driver.
- Calling other Cadence driver functions as required, either from driver framework functions or stack functions.

2.5. Concurrency Models

Cadence Core Drivers are architected such that they are portable to platforms which enforce a particular concurrency model, while also giving you the maximum freedom to choose the model suitable for your application. Also, the minimal assumptions made allow for maximum efficiency on simple platforms. Part of the API for most drivers

involves calling back your code. The basic rule is that these callbacks are made from the driver's interrupt handler. This is what yields the freedom and portability just mentioned.

Chapter 3. Common Driver Interface

3.1. Overview

Each Cadence Core Driver represents a different technology and accordingly, offers varying functionality. However, all Cadence drivers share a common framework of functions, to quickly get you beyond the basic bringup and on to making use of the Cadence technology you have acquired.

The following functions are common to all Cadence drivers:

- `probe(configurationInfo, resourceRequirements)` - this is the first function you should call. Although the exact signature varies by driver, the pattern is that you pass information about how the driver should be used, and it fills information about the required resources (RAM, locks) to support that configuration. In a dynamic system or where your code is to be ported to a variety of platforms, you may wish to call probe repeatedly to "negotiate" an acceptable configuration that fits within platform resources. The configuration includes the base address of the hardware's registers, so the driver may read identification from the hardware during this call (to determine resource requirements).

- `init(instanceData, configurationInfo)` - after a successful probe(), call this function, passing a pointer to an allocated block of RAM of the required size (the required size is filled by probe), and the configuration again (so the driver may copy anything it must remember).

- `start(instanceData)` - after a successful init, you should attach the driver's interrupt handler (isr) and enable the interrupt at the interrupt controller. Then, call this function to start the driver. It will enable relevant interrupts at the source.

- `isr(instanceData)` - this is the driver's interrupt handler, which your code must attach before calling start()

- `stop(instanceData)` - this will disable any hardware interrupts at the source

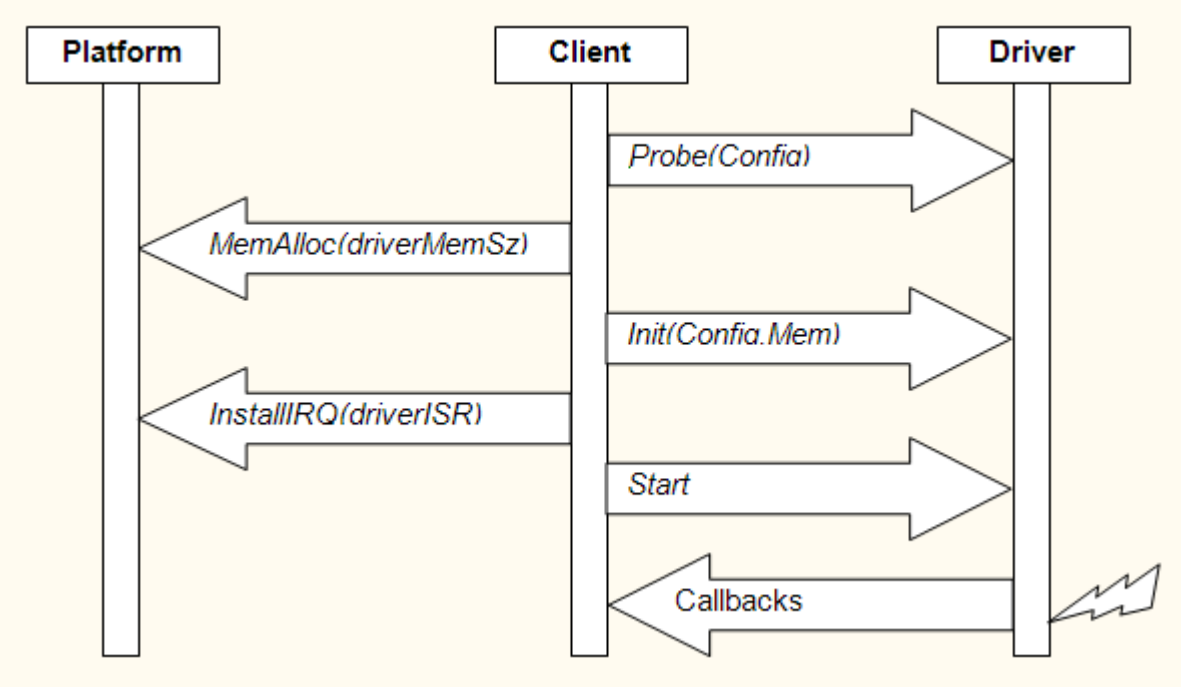
- `destroy(instanceData)` - this will internally stop, and put the driver in a non-responsive state until init is called again

The instanceData parameter is a pointer to a RAM block, allocated by the client before calling `init()`, based on the resource requirements filled by `probe()`. The reason for this mechanism is to allow for multiple instances of the driver, in case you have more than one of the same type of Cadence controller in your system.

3.2. Driver Initialization & Startup

The common aspects of driver initialization and startup are illustrated below:

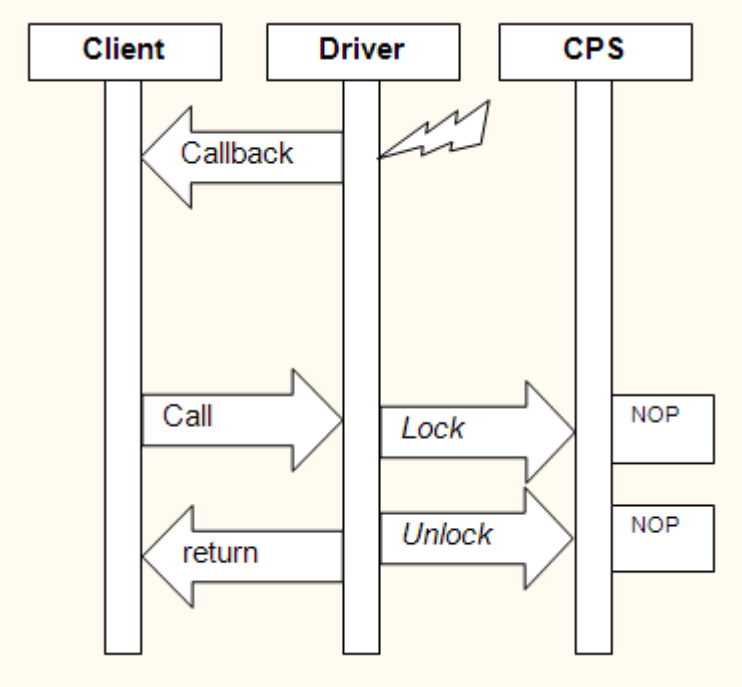
Figure 3.1. Driver Initialization & Startup Sequence



3.3. Concurrency Model Details

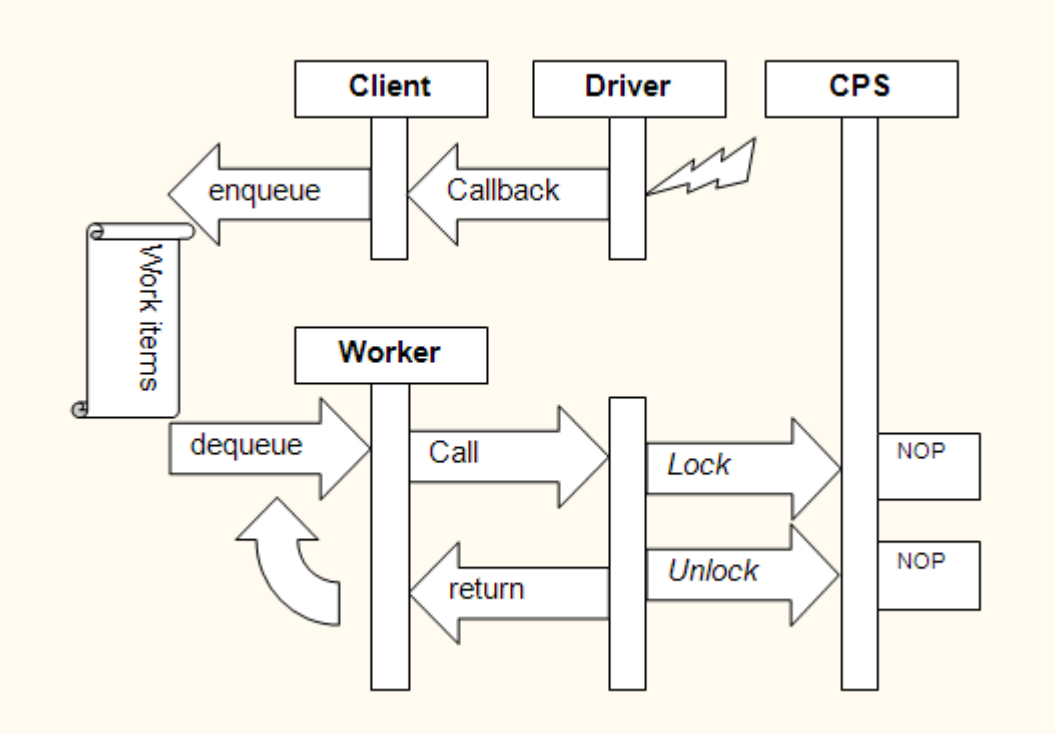
Due to the minimal assumptions made by Cadence drivers, you can choose the concurrency model best for your system or application.

Figure 3.2. Pure IRQ Concurrency Model



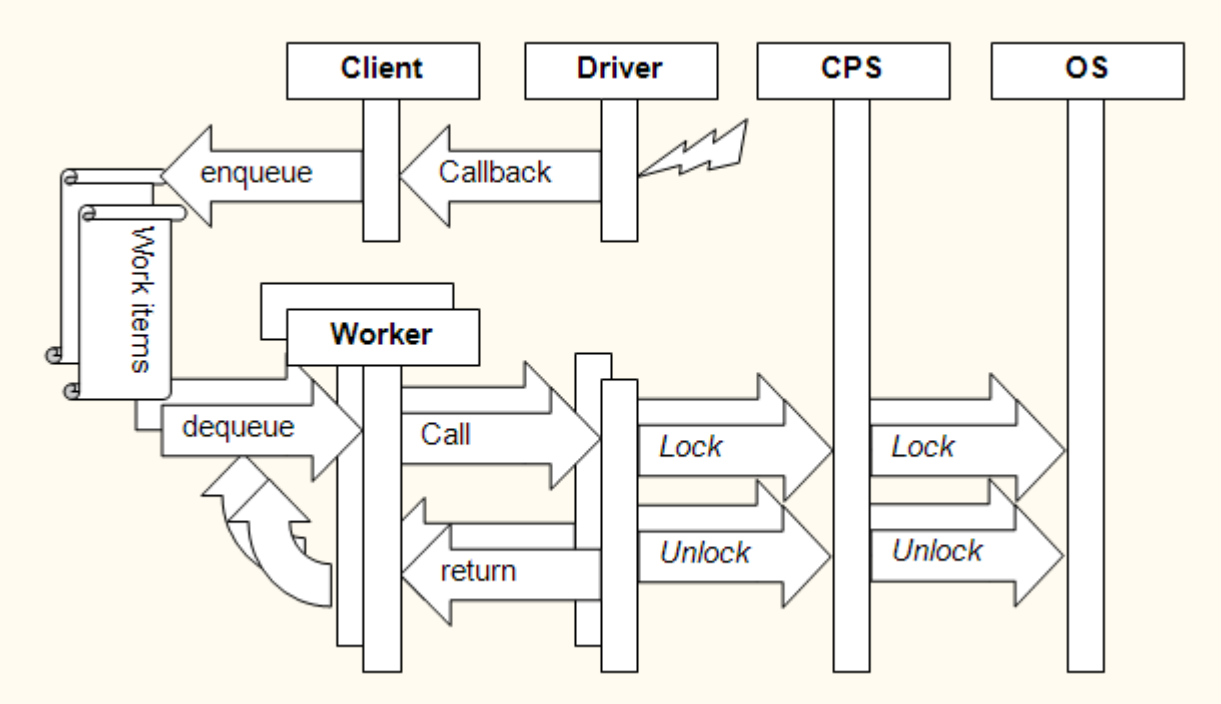
This simple model executes your callback handlers within the ISR. It may be appropriate for a deeply-embedded system which does not need to handle other tasks while servicing driver callbacks, or where the callbacks execute quickly. There is no need to implement CPS locks for this model - they can be "NOP."

Figure 3.3. Single Worker Concurrency Model



This model executes your callbacks in a single task, "upper half" or whatever is appropriate to your operating system. Even though driver data is accessed by the ISR and the worker in this model, Cadence drivers are constructed to eliminate the need for locks in this scenario. So, there is no need to implement CPS locks for this model - they can be "NOP."

Figure 3.4. Multiple Workers Concurrency Model



This model executes your callbacks in multiple tasks, kernel threads, or whatever is appropriate to your operating system. In this case, CPS locks must be implemented, because the same driver data can be read/written in multiple contexts.

Chapter 4. Cadence Platform Services Reference

4.1. Introduction

The chapter documents the functions that are required to be implemented relative to the platform for the driver to access the hardware resources.

4.2. CPS_LockHandle

Function Declaration

```
void * CPS_LockHandle
```

Parameter List

Description

A lock handle.

4.3. CPS_ProbeLocks

Function Declaration

```
uint32_t CPS_ProbeLocks (uint32_t lockCount)
```

Parameter List

lockCount [in]

number of locks requested

Description

Check that sufficient locks are available.

Return Value

0 on success (locks available)

ENOENT if insufficient locks are available

4.4. CPS_InitLock

Function Declaration

```
uint32_t CPS_InitLock (CPS_LockHandle *lock)
```

Parameter List

lock [out]

where to store the allocated, initialized lock

Description

Initialize a lock.

Return Value

0 on success (lock is allocated and initialized)

ENOENT if insufficient locks are available

4.5. CPS_FreeLock

Function Declaration

```
void CPS_FreeLock (CPS_LockHandle lock)
```

Parameter List

lock [in]

the lock

Description

Free a lock.

4.6. CPS_Lock

Function Declaration

```
uint32_t CPS_Lock (CPS_LockHandle lock)
```

Parameter List

lock [in]

the lock

Description

Lock a lock, pending the current thread/task if necessary until the lock is available.

4.7. CPS_Unlock

Function Declaration

```
uint32_t CPS_Unlock (CPS_LockHandle lock)
```

Parameter List

lock [in]

the lock

Description

Unlock a lock, readying the next highest-priority thread/task pended on it if any.

4.8. CPS_UncachedRead8

Function Declaration

```
uint8_t CPS_UncachedRead8 (volatile uint8_t *address)
```

Parameter List

address [in]

the address

Description

Read a byte, bypassing the cache.

Return Value

the byte at the given address

4.9. CPS_UncachedRead16

Function Declaration

```
uint16_t CPS_UncachedRead16 (volatile uint16_t *address)
```

Parameter List

address [in]

the address

Description

Read a short, bypassing the cache.

Return Value

the short at the given address

4.10. CPS_UncachedRead32

Function Declaration

```
uint32_t CPS_UncachedRead32 (volatile uint32_t *address)
```

Parameter List

address [in]

the address

Description

Read a (32-bit) word, bypassing the cache.

Return Value

the word at the given address

4.11. CPS_UncachedWrite8

Function Declaration

```
void CPS_UncachedWrite8 (volatile uint8_t *address, uint8_t value)
```

Parameter List

address [in]

the address

value [in]

the byte to write

Description

Write a byte to memory, bypassing the cache.

4.12. CPS_UncachedWrite16

Function Declaration

```
void CPS_UncachedWrite16 (volatile uint16_t *address, uint16_t value)
```

Parameter List

address [in]

the address

value [in]

the short to write

Description

Write a short to memory, bypassing the cache.

4.13. CPS_UncachedWrite32

Function Declaration

```
void CPS_UncachedWrite32 (volatile uint32_t *address, uint32_t value)
```

Parameter List

address [in]

the address

value [in]

the word to write

Description

Write a (32-bit) word to memory, bypassing the cache.

4.14. CPS_WritePhysAddress32

Function Declaration

```
void CPS_WritePhysAddress32 (volatile uint32_t *location, uint32_t addrValue)
```

Parameter List

location [in]

the (CPU) location where to write the address value

addrValue [in]

the address value to write

Description

Write a (32-bit) address value to memory, bypassing the cache.

This function is for writing an address value, i.e. something that will be treated as an address by hardware, and therefore might need to be translated to a physical bus address.

4.15. CPS_BufferCopy

Function Declaration

```
void CPS_BufferCopy (volatile uint8_t *dst, volatile uint8_t *src, uint32_t
size)
```

Parameter List

src [in]

src address

dst [in]

destination address

size [in]

size of the copy

Description

Hardware specific memcpy.

4.16. CPS_CacheInvalidate

Function Declaration

```
void CPS_CacheInvalidate (uintptr_t address, size_t size, uintptr_t devInfo)
```

Parameter List

address [in]

Virtual address of memory region. (If an MMU is not in use, this will be equivalent to the physical address.) This address should be rounded down to the nearest cache line boundary.

size [in]

size of memory in bytes. This size should be rounded up to the nearest cache line boundary. Use size UINTPTR_MAX to invalidate all memory cache. A size of 0 should be ignored and the function should return immediately with no effect.

devInfo [in]

This parameter can be used to pass implementation specific data to this function. The content and use of this parameter is up to the implementor of this function to determine, and if not required it may be ignored. For example, under Linux it can be used to pass a pointer to the device struct to be used in a call to dma_sync_single_for_device(). If used, the parameter should be passed to the core driver at initialisation as part of the configurationInfo struct. Please see the core driver documentation for details of how to do this.

Description

Invalidate the cache for the specified memory region.

This function may be stubbed out if caching is disabled for memory regions as described in the driver documentation, or if the driver configuration does not require this function.

4.17. CPS_CacheFlush

Function Declaration

```
void CPS_CacheFlush (uintptr_t address, size_t size, uintptr_t devInfo)
```

Parameter List

address [in]

Virtual address of memory region. (If an MMU is not in use, this will be equivalent to the physical address.) This address should be rounded down to the nearest cache line boundary.

size [in]

size of memory in bytes. This size should be rounded up to the nearest cache line boundary. Use size UINTPTR_MAX to flush all memory cache. A size of 0 should be ignored and the function should return immediately with no effect.

devInfo [in]

This parameter can be used to pass implementation specific data to this function. The content and use of this parameter is up to the implementor of this function to determine, and if not required it may be ignored. For example, under Linux it can be used to pass a pointer to the device struct to be used in a call to dma_sync_single_for_device(). If used, the parameter should be passed to the core driver at initialisation as part of the configurationInfo struct. Please see the core driver documentation for details of how to do this.

Description

Flush the cache for the specified memory region This function may be stubbed out if caching is disabled for memory regions as described in the driver documentation, or if the driver configuration does not require this function.

Chapter 5. Sample CPS implementation for Bare Metal

5.1. Overview

The code here implements the Cadence Porting System for a bare-metal environment. This should be applicable to most bare-metal systems where there is no MMU or data cache.

5.2. Sample code

```
uint8_t CPS_UncachedRead8(volatile uint8_t* address) {
    return *address;
}

uint16_t CPS_UncachedRead16(volatile uint16_t* address) {
    return *address;
}

uint32_t CPS_UncachedRead32(volatile uint32_t* address) {
    return *address;
}

void CPS_UncachedWrite8(volatile uint8_t* address, uint8_t value) {
    *address = value;
}

void CPS_UncachedWrite16(volatile uint16_t* address, uint16_t value) {
    *address = value;
}

void CPS_UncachedWrite32(volatile uint32_t* address, uint32_t value) {
    *address = value;
}

void CPS_BufferCopy(volatile uint8_t *dst, volatile uint8_t *src, uint32_t size) {
    memcpy((void*)dst, (void*)src, size);
}

uint32_t CPS_ProbeLocks(uint32_t lockCount) {
    return 0;
}

uint32_t CPS_InitLock(CPS_LockHandle* lock) {
    return 0;
}

void CPS_FreeLock(CPS_LockHandle lock) {
    return;
}

uint32_t CPS_Lock(CPS_LockHandle lock) {
    return 0;
}

uint32_t CPS_Unlock(CPS_LockHandle lock) {
    return 0;
}

void CPS_CacheInvalidate(uintptr_t address, size_t size, uintptr_t devInfo) {
    return;
}
```

```
void CPS_CacheFlush(uintptr_t address, size_t size, uintptr_t devInfo) {
    return;
}
```

Chapter 6. Sample CPS implementation for Linux

6.1. Overview

The code here implements the Cadence Porting System for a Linux environment. This should be applicable to Linux kernels from 2.6.32 onwards with minor modifications for your system. Note that locks are not implemented in this sample.

If a Cadence Driver makes use of the CacheFlush() and CacheInvalidate() functions the driver documentation will provide details of how to pass the "device" pointer to the Core Driver so that it will be available to your implementation of CPS.

6.2. Sample code

```
uint8_t CPS_UncachedRead8(volatile uint8_t* address) {
    return readb(address);
}

uint16_t CPS_UncachedRead16(volatile uint16_t* address) {
    return readw(address);
}

uint32_t CPS_UncachedRead32(volatile uint32_t* address) {
    return readl(address);
}

void CPS_UncachedWrite8(volatile uint8_t* address, uint8_t value) {
    writeb (value, address);
}

void CPS_UncachedWrite16(volatile uint16_t* address, uint16_t value) {
    writew (value, address);
}

void CPS_UncachedWrite32(volatile uint32_t* address, uint32_t value) {
    writel (value, address);
}

void CPS_BufferCopy(volatile uint8_t *dst, volatile uint8_t *src, uint32_t size) {
    memcpy((void*)dst, (void*)src, size);
}

uint32_t CPS_ProbeLocks(uint32_t lockCount) {
    return 0;
}

uint32_t CPS_InitLock(CPS_LockHandle* lock) {
    return 0;
}

void CPS_FreeLock(CPS_LockHandle lock) {
    return;
}

uint32_t CPS_Lock(CPS_LockHandle lock) {
    return 0;
}

uint32_t CPS_Unlock(CPS_LockHandle lock) {
    return 0;
```

```
}

/**
 * Invalidate cache using Linux DMA sync routines
 * @param[in] address Virtual address of buffer to sync. Buffer must
 *    be allocated using dma_map_single() function
 * @param[in] size Size of buffer
 * @param[in] device Pointer to struct device* which owns buffer
 */
void CPS_CacheInvalidate(uintptr_t address, size_t size, uintptr_t device) {
    uintptr_t phys_address = __pa(address);
    struct device *dev = (struct device *) device;
    dma_sync_single_for_cpu(dev, phys_address, size, DMA_BIDIRECTIONAL);
}

/**
 * Flush cache using Linux DMA sync routines
 * @param[in] address Virtual address of buffer to sync. Buffer must
 *    be allocated using dma_map_single() function
 * @param[in] size Size of buffer
 * @param[in] device Pointer to struct device* which owns buffer
 */
void CPS_CacheFlush(uintptr_t address, size_t size, uintptr_t device) {
    uintptr_t phys_address = __pa(address);
    struct device *dev = (struct device *) device;
    dma_sync_single_for_device(dev, phys_address, size, DMA_BIDIRECTIONAL);
}
```